



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

Introducción a HPC para IA

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Tabla de Contenidos

- 1 Método de evaluación y calificaciones
- 2 Introducción
- 3 Fundamentos de arquitecturas computacionales
- 4 Métricas de rendimiento y eficiencia en HPC para IA
- 5 System-on-chip (SoC)
- 6 Modelos de programación paralela
- 7 Desafíos de HPC en IA
- 8 Inference Optimizer

Método de evaluación y calificaciones

- ▶ Sección 1: Fundamentos de Computación de Alto Rendimiento
 - Introducción a los sistemas de alto rendimiento (HPC)
 - Arquitecturas: CPU, GPU, FPGA, ASIC
 - Introducción básica a programación de GPU con CUDA
 - Frameworks y despliegue: **ONNX**, **TensorFlow Lite**, **Torch**
 - Aceleración de inferencia con **TensorRT**
- ▶ Sección 2: Optimización e Inferencia de Modelos de IA
 - **Hardware Description Language (HDL)**, **High-level Synthesis (HLS)**
 - Introducción a **Vitis AI** para FPGAs y SoCs
- ▶ Sección 3: Aplicaciones e Implementación
 - Diseño e implementación de pipelines de inferencia
 - Comparación de rendimiento: CPU vs GPU vs Edge
 - Despliegue en plataformas embebidas: **Jetson AGX Orin**, **Jetson ORIN Nano**, **Raspberry Pi**, **ESP32**, **Zynq**
 - Optimización de modelos para ejecución en tiempo real - profiling - benchmarking.

► **Laboratorios y/o tareas (100 %):**

- Una o dos actividades calificadas.
- Trabajo individual.
- Informe de laboratorio, códigos, documentación, presentaciones, lecturas, entre otros.

-
- Sección 2 - **Gabriel Saavedra (UdeC)**
- Sección 3 - **Hugo Gárces (UdeC)**
- Sección 4 - **Crithian Aguilera (UBB)**

¿Qué se espera que aprendan?

► Técnico.

- La integración de conocimientos teóricos y de codificación a nivel algorítmico para resolver problemas científicos y de ingeniería implementando pipelines de inferencia de algoritmos de DL mediante la experiencia práctica..
- **Sistemas embebidos, computación del alto rendimiento y optimizadores de inferencia.**

► No técnico.

- Trabajo autónomo.
- Disciplina.
- Búsqueda de información.
- Pensamiento crítico.
- Resolución de problemas y proactividad.
- Trabajar con poca orientación.

Organización de las semanas

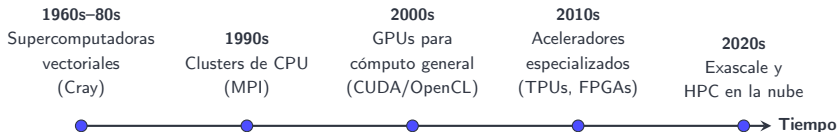


Introducción

- ▶ **Computación de Alto Rendimiento (HPC)** se refiere al uso de sistemas con una gran capacidad de procesamiento para resolver problemas complejos que no pueden ser abordados por computadoras convencionales.
- ▶ Incluye supercomputadoras, clústeres de GPUs, FPGAs y arquitecturas paralelas.
- ▶ En IA moderna, especialmente en Deep Learning:
 - Los modelos tienen millones o billones de parámetros.
 - El entrenamiento puede tomar días o semanas sin aceleración.
 - La inferencia en tiempo real requiere baja **latencia** y alto **throughput**.

- ▶ **Computación de Alto Rendimiento (HPC)** se refiere al uso de sistemas con una gran capacidad de procesamiento para resolver problemas complejos que no pueden ser abordados por computadoras convencionales.
- ▶ Incluye supercomputadoras, clústeres de GPUs, FPGAs y arquitecturas paralelas.
- ▶ **En IA moderna, especialmente en Deep Learning:**
 - Los modelos tienen millones o billones de parámetros.
 - El entrenamiento puede tomar días o semanas sin aceleración.
 - La inferencia en tiempo real requiere baja **latencia** y alto **throughput**.
- ▶ **HPC permite:**
 - Acelerar el entrenamiento de modelos (usando GPUs/TPUs).
 - Optimizar la inferencia en edge devices (Jetson, FPGA, etc.).
 - Escalar soluciones para producción en tiempo real.

Tendencias y Contexto Histórico de HPC en IA



Tendencias:

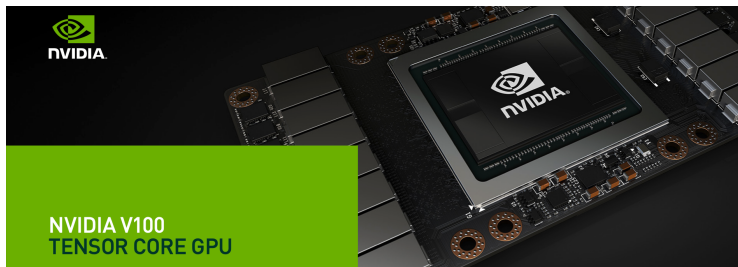
- ▶ **Exascale Computing:** 10^{18} ops/seg¹.
- ▶ **Green AI:** optimización energética (*perf/Watt*)².
- ▶ HPC en la nube: AWS, Azure, Google Cloud.
- ▶ IA en Edge: despliegue en dispositivos embebidos.

¹D. Schneider, "The Exascale Era is Upon Us: The Frontier supercomputer may be the first to reach 1,000,000,000,000,000,000 operations per second," in IEEE Spectrum, vol. 59, no. 1, pp. 34-35, January 2022.

²Verdecchia, R., Sallou, J., Cruz, L. (2023). A systematic review of Green AI. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 13(4), e1507.

Motivación: Entrenamiento de GPT en supercomputadoras.

- ▶ Los modelos de lenguaje como **GPT** requieren enormes cantidades de cómputo para entrenarse.
- ▶ Ejemplo: **GPT-3** (175mil millones de parámetros)
 - ~3000 GPUs A100 durante semanas
 - Cientos de millones de dólares en costos de entrenamiento
 - Consumo energético comparable al de una ciudad pequeña
- ▶ HPC (<https://www.nvidia.com/es-la/data-center/v100/>)
 - Uso de miles de GPUs en paralelo
 - Alto rendimiento para mover grandes volúmenes de datos.
 - Escalabilidad eficiente, entrenamiento distribuido en clústeres con SLURM

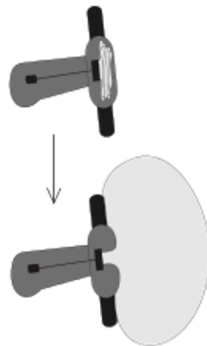


- ▶ Sistema informático donde la precisión depende tanto de los **aspectos temporales** como funcionales de los cálculos³.
- ▶ Las tareas ejecutadas están sujetas a **estrictas limitaciones de tiempo** (deadlines).
 - Un resultado correcto obtenido en el momento equivocado puede tener **consecuencias catastróficas** para el usuario o el entorno.

³Salem, M., Carvajal, G., Liu, T., Fischmeister, S. (2019). Assessing the Robustness of Arrival Curves Models for Real-Time Systems. In: André, É., Stoelinga, M. (eds) Formal Modeling and Analysis of Timed Systems. FORMATS 2019. Lecture Notes in Computer Science(), vol 11750. Springer, Cham.

https://doi.org/10.1007/978-3-030-29662-9_2

¿Por qué nos preocupan los delays?



- ▶ La decisión de activar un airbag debe tomarse en un plazo de 15 a 30 ms.
- ▶ Un retraso adicional de 10 ms por encima del plazo inutilizará el airbag.

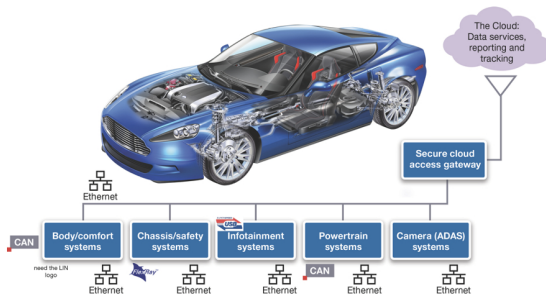
- ▶ Muchos sistemas actuales requieren **respuestas en tiempo real** con recursos limitados:
 - Vehículos autónomos
 - Cámaras inteligentes para seguridad
 - Drones para reconocimiento en campo
- ▶ Desafíos:
 - Procesamiento local de datos de sensores (imagen, señal, etc.)
 - **Latencia mínima: decisiones en milisegundos.**
 - **Restricciones de energía y capacidad computacional.**
- ▶ HPC permite:
 - Optimización de modelos mediante técnicas como **quantization** y **pruning**.
 - Entrenamiento distribuido en clústeres.
 - Simulación y validación de rendimiento en entornos controlados.



- ▶ Múltiples subsistemas que interactúan y comparten resultados limitados (el software subyacente se vuelve más complejo).
- ▶ Condiciones operativas dinámicas (por ejemplo, variaciones en el suministro eléctrico, degradación del hardware, etc.).
- ▶ Por ejemplo, en un vehículo aéreo no tripulado típico, hay múltiples procesadores integrados que deben coordinar las acciones de sensores y actuadores distribuidos en múltiples subsistemas, que pueden tener patrones computacionales muy diferentes.

Sistemas de tiempo real modernos

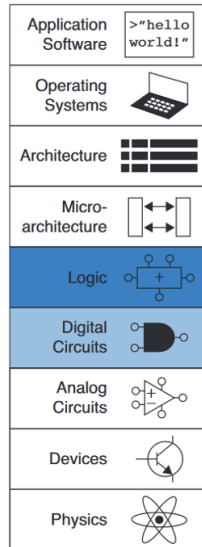
- ▶ Muchos subsistemas interactuando y compartiendo resultados limitados.
- ▶ This Car Runs on Code, by R. Charette in IEEE Spectrum (February 2009).
 - 100+ distributed embedded computers.
 - Kilometers of internal wiring.
 - 300M+ lines of code.



Edge AI es el uso de algoritmos y modelos de IA directamente en dispositivos cercanos a donde se generan los datos, sin depender de enviar la información a la nube o a un centro de datos para su procesamiento.

Fundamentos de arquitecturas computacionales

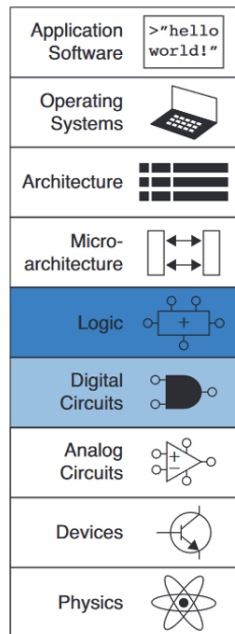
Capas de abstracción



Computación de propósito general

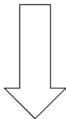


- Dígale a un sistema preconstruído qué hacer (CPU, Many-Cores, GPUs).
- Tenemos que jugar con lo que tenemos aquí.



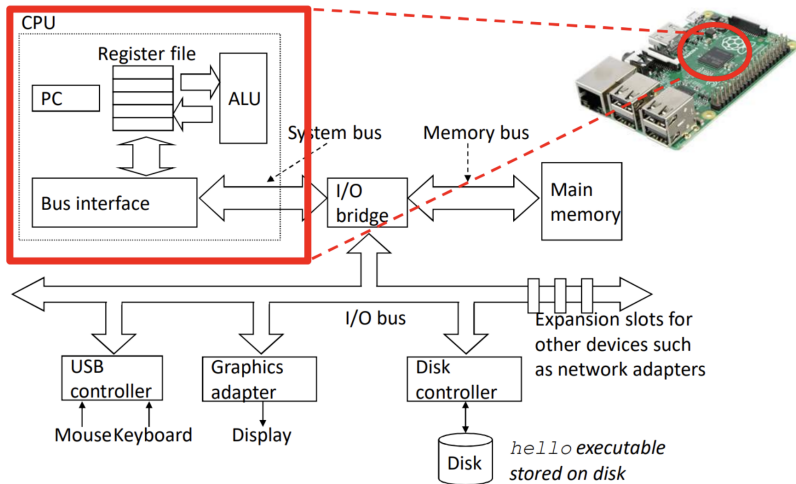
```
int a,b,c,d;  
printf("hello world~\n");  
d= a + b * c;  
printf("%i", d);
```

Application
(software)

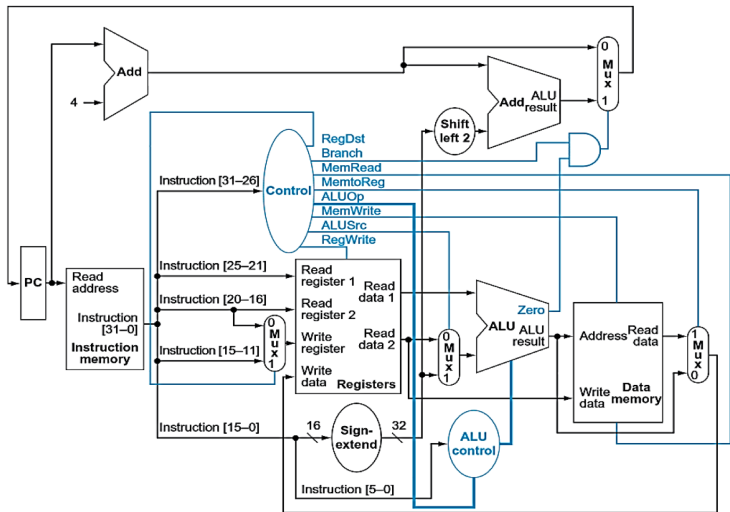


Platform
(hardware)

Organización típica de un computador



Una vista simplificada de una CPU



PC (Program Counter) guarda la dirección de la instrucción actual.

PC + 4 (32 bits), ALUOp -> 00: Sum, 01:Res, etc,

CPU: Núcleo de ejecución secuencial en arquitecturas de propósito general

- ▶ Procesador central encargado de ejecutar programas en computadoras.
- ▶ **Necesita componentes externos (RAM, almacenamiento, etc.).**
- ▶ Diseñado para tareas generales y sistemas complejos.
- ▶ Alto rendimiento, velocidades superiores (GHz).
- ▶ Limitaciones de rendimiento derivadas de la naturaleza secuencial del procesamiento: techo de ILP (Instruction-Level Parallelism)⁴, ley de Amdahl⁵, y cuellos de botella del sistema de memoria (memory wall).
- ▶ **Escalabilidad física restringida por la Ley de Dennard, estancamiento de la frecuencia de reloj, y desafíos térmicos asociados al diseño de silicio en nodos sub-7nm.**
- ▶ **Desplazamiento parcial hacia arquitecturas heterogéneas (GPGPU, ASICs, TPUs) en respuesta al agotamiento de las ganancias tradicionales según la Ley de Moore.**
- ▶ **Métricas clave de evaluación: instrucciones por ciclo (IPC), eficiencia energética (perf/-Watt), latencia, y throughput bajo cargas de trabajo paralelas y mixtas.**

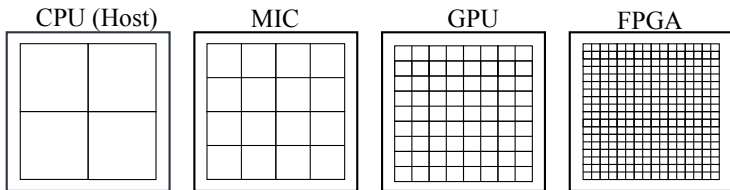
⁴capacidad de un procesador para ejecutar múltiples instrucciones al mismo tiempo aprovechando que no todas dependen unas de otras

⁵establece que la ganancia máxima de velocidad al paralelizar un programa está limitada por la fracción de éste que debe ejecutarse de forma secuencial.

CPU-only Applications



Todo está en la memoria de CPU y ejecutado secuencial o multihilo tradicional.



- ▶ La *Central Processing Unit* (CPU) convencional, junto con cualquier combinación de aceleradores especializados que ofrecen granularidad mixta para procesamiento paralelo⁶.
- ▶ Los aceleradores especializados incluyen:
 - **Many Integrated Cores (MIC)** processors⁷.
 - **Graphics Processing Units (GPUs)**.
 - **Field-Programmable Gate Arrays (FPGAs)**.
- ▶ Estas tecnologías representan la mayor parte de la innovación en hardware disponible hoy en día para aplicaciones de ML.

⁶La unidad de cómputo es capaz de dividir las tareas en porciones más pequeñas y procesarlas en paralelo.

⁷Intel Xeon Phi, son procesadores diseñados para el cómputo paralelo masivo. Su arquitectura se basa en múltiples cores para procesar tareas de manera simultánea.

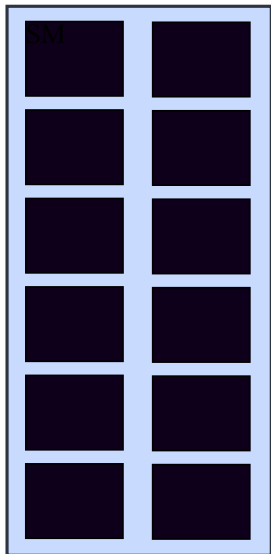
Procesador orientado a paralelismo masivo de datos

- ▶ **Arquitectura orientada a throughput:** basada en múltiples **Streaming Multi-processors (SMs)** con decenas de miles de hilos concurrentes bajo el modelo SIMT (Single Instruction, Multiple Threads).
- ▶ Optimizadas para cargas de trabajo con alto paralelismo a nivel de datos: **gráficos, aprendizaje profundo, simulaciones numéricas y álgebra lineal.**
- ▶ Estructura jerárquica de memoria: registros por hilo, memoria compartida por bloque, **cachés L1/L2** y memoria global; gestión explícita de la localidad por parte del programador (CUDA/OpenCL).
- ▶ Rendimiento altamente sensible a la divergencia de hilos dentro de warps (unidades de ejecución SIMT).⁸
- ▶ **Limitaciones**, consumo energético elevado bajo carga plena, restricciones de ancho de banda, y menor eficiencia en tareas con poca paralelización o fuertes dependencias secuenciales.

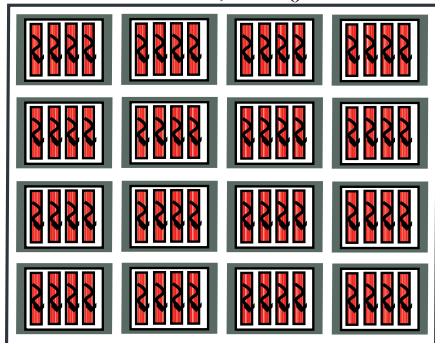
*Líneas de investigación: arquitecturas de próxima generación para ML, escalado multinodo y eficiencia energética en exaescala.*_____

⁸cuando dentro de un warp algunos hilos entran a un if y otros al else. En ese caso, la GPU no puede ejecutar ambas ramas a la vez, así que ejecuta en serie cada camino desactivando temporalmente los hilos que no aplican.. esto reduce el paralelismo y afecta el rendimiento.

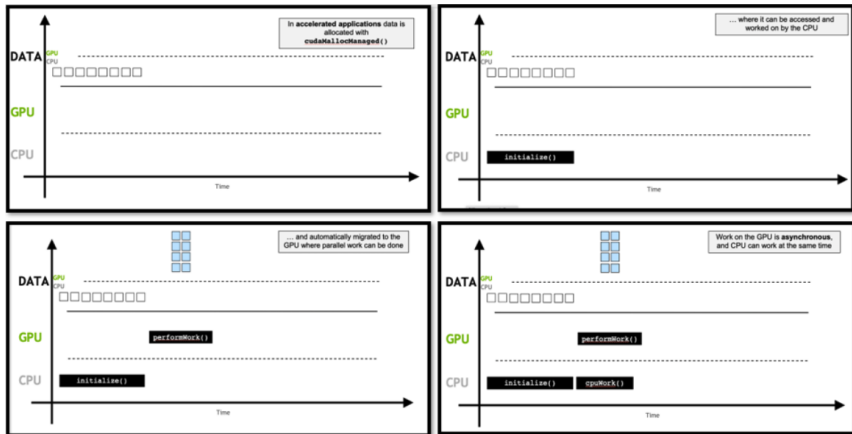
GPU



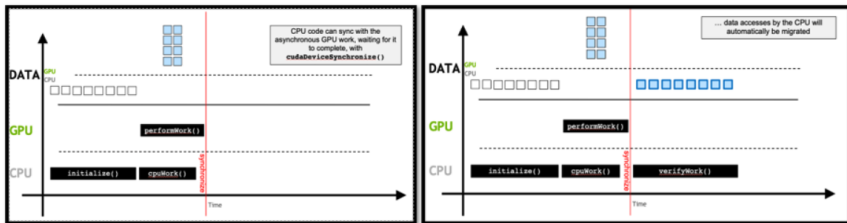
```
functionwork<<<16, 4>>>()>>>()
```



GPU-accelerated



⁹X. Chen, G. S. Gutmann, and J. Bungio, "Deep Learning by Doing: The NVIDIA Deep Learning Institute and University Ambassador Program," *Journal of Computational Science Edu*

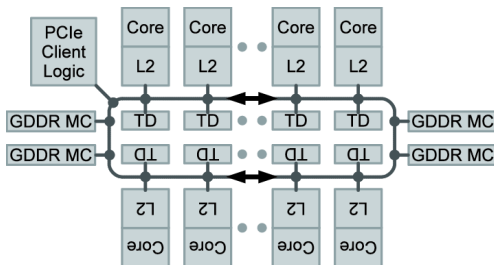


- ▶ Se aprovecha la ejecución en paralelo masiva de la GPU.
- ▶ La memoria unificada facilita el desarrollo sin necesidad de copias explícitas.
- ▶ Se puede lograr ejecución asincrónica para mayor eficiencia.

¹⁰X. Chen, G. S. Gutmann, and J. Bungo, "Deep Learning by Doing: The NVIDIA Deep Learning Institute and University Ambassador Program," *Journal of Computational Science Edu*

SIMD (Single Instruction, Multiple Data):

- ▶ Una única instrucción ejecuta una instrucción sobre múltiples datos al mismo tiempo.
- ▶ Ejemplo: suma de dos vectores elemento a elemento.
- ▶ Común en GPUs y operaciones vectorizadas.



- ▶ **MIMD**: múltiples instrucciones sobre múltiples datos (ej. CPU multinúcleo como Intel Xeon Phi).
- ▶ **Multihilo**: múltiples hilos que comparten recursos para ejecutar en paralelo.
- ▶ **Vectorización**: técnica que permite aplicar SIMD automáticamente al [compilar código](#).

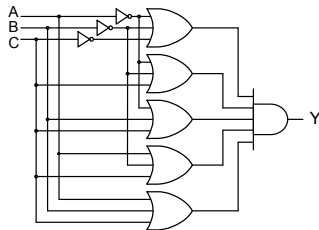
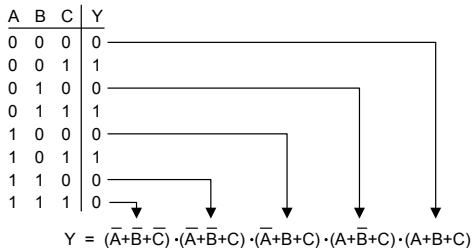
¹¹GDDR MC: controlador de memoria GDDR5 y TD: tag directoy, memoria cache.

► Especificación

- Tablas de verdad
- Diagramas de estado

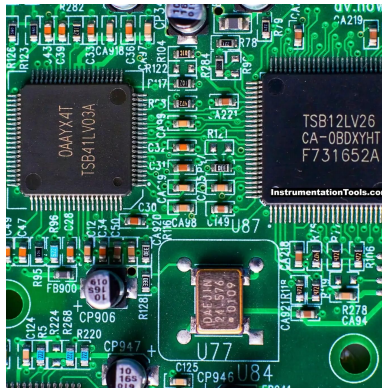
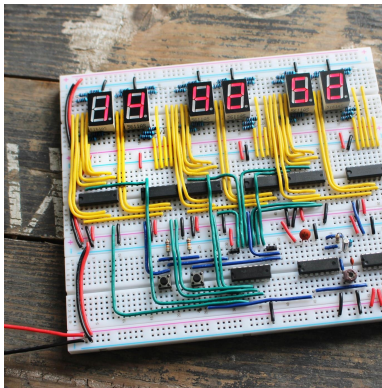
► Diseño

- Optimización local y traducción manual a puertas lógicas
- Captura esquemática

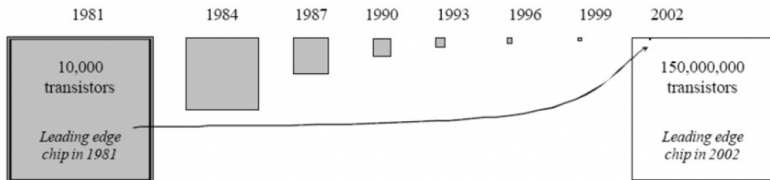


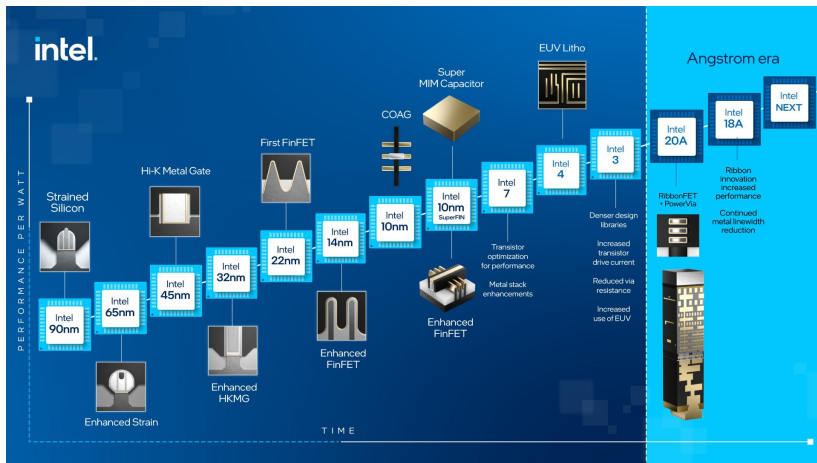
► Implementación

- Mapeo a puertas discretas
- Prototipo, pruebas y depuración (placa de pruebas)
- Implementación final

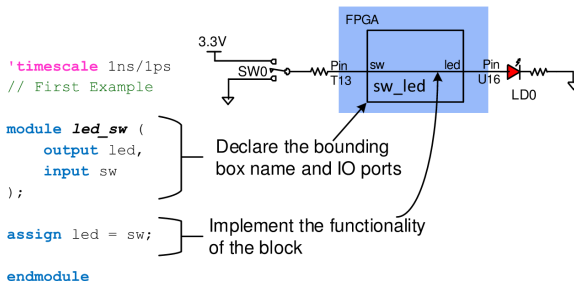


- ▶ Proceso largo y tedioso, propenso a errores.
- ▶ Documentación deficiente.
- ▶ En los años 90, el crecimiento exponencial de la complejidad hizo inviable la captura esquemática.
 - *Un circuito integrado fabricado en 2002 podía asignar 15 000 chips con tecnología de vanguardia en 1980.*





Diseño de sistemas digitales (hoy en día)



► Lenguajes de descripción de hardware (HDL)

- Abstracción para describir la estructura y el comportamiento de circuitos digitales.
- Más legibles para humanos.
- Las herramientas de síntesis EDA¹³ automatizan la traducción de una descripción HDL a netlists.
- Herramientas adicionales realizan el "place and routing" para definir la implementación física
- **No están ligados a una tecnología específica**

Dos sabores principales de HDL

VHDL:	Verilog:
2 process ((S0,S1),A,B,C,D) 1	
3 begin 2	
4 case (S0,S1), is 3	always @((S0,S1), A, B, C, D)
5 when "00" => Y <= A; 4	case ((S0,S1))
6 when "01" => Y <= B; 5	2'b00: Y = A;
7 when "10" => Y <= C; 6	2'b01: Y = B;
8 when "11" => Y <= D; 7	2'b10: Y = C;
9 when others => Y <= A; 8	2'b11: Y = D;
10 end case; 9	endcase
11 end process; 10	

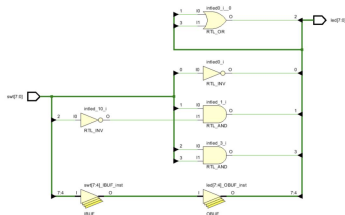
► Verilog (Verification Logic) and System Verilog

- Tipado débil
- Conciso
- Sintaxis similar a C
- Solo es determinista si se siguen buenas prácticas

► VHDL

- Tipado fuerte
- Verboso
- Sintaxis poco familiar para programadores tradicionales
- Determinista
- Curva de aprendizaje empinada

```
module logic_design (  
    input  logic [7:0] swt,  
    output logic [7:0] led  
);  
  
    // Operaciones lógicas individuales  
    assign led[0] = ~swt[0];  
    assign led[1] = swt[0] & swt[1];  
    assign led[2] = ~swt[2] | swt[1];  
    assign led[3] = swt[2] & swt[3];  
  
    // Asignación directa con buffers a los LEDs 7:4  
    assign led[7:4] = swt[7:4];  
  
endmodule
```

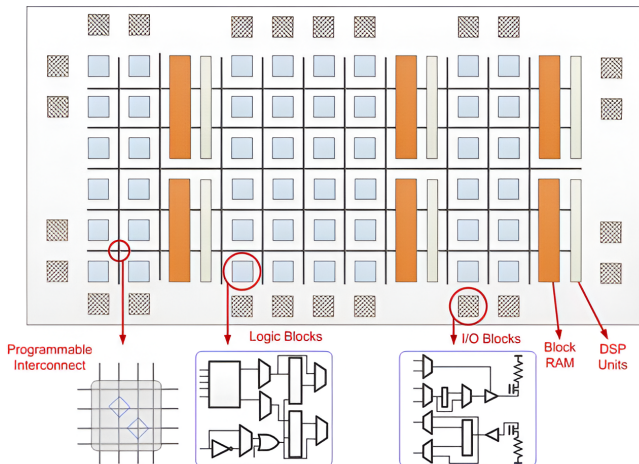


► **¡Los HDL no son lenguajes de programación!**

- Estás describiendo un sistema
- No se compila un código HDL
- No se ejecuta un código HDL

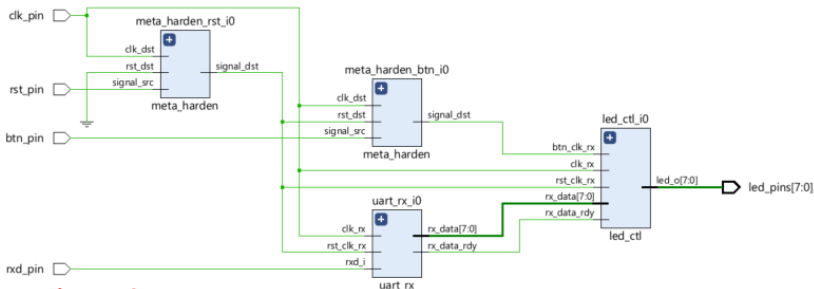
► **¡El software no obedece las leyes de la física, el hardware sí!**

Basics of Field-Programmable Gate Arrays (FPGA)



- Equivalente a millones de puertas lógicas discretas encapsuladas en un chip diminuto.
- ¿Cómo gestionar la complejidad?

Jerarquía, Modularidad y Regularidad



- **Jerarquía**

Divide un sistema complejo en módulos, y luego subdivide esos módulos hasta que las piezas sean fáciles de entender.

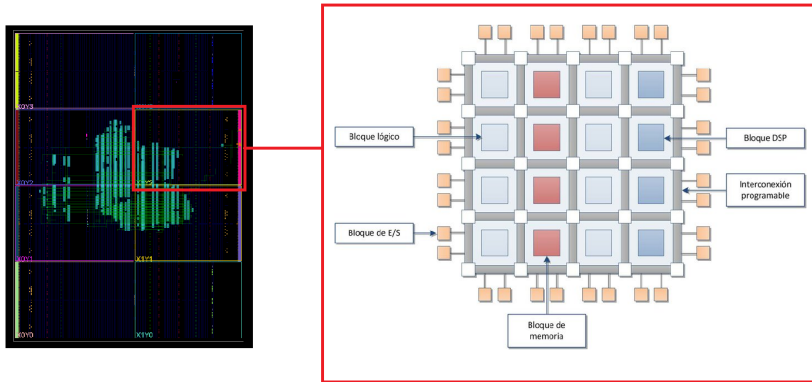
- **Modularidad**

Los módulos deben tener funciones e interfaces bien definidas, de forma que puedan conectarse fácilmente sin efectos inesperados.

- **Regularidad**

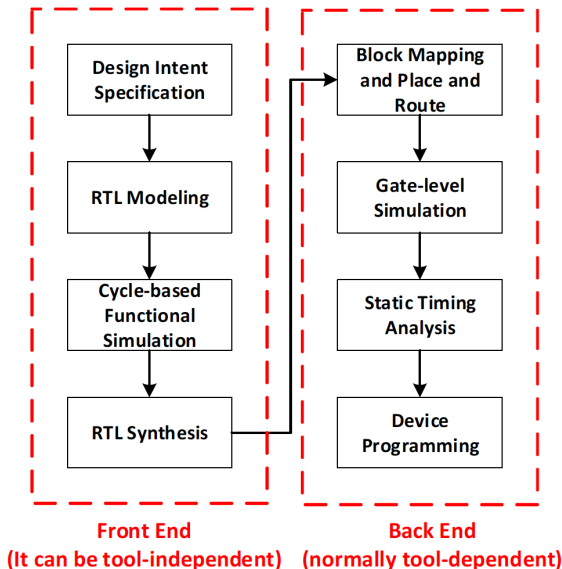
Promueve la uniformidad entre módulos, permitiendo que los módulos comunes se reutilicen muchas veces.

What is inside? (FPGA)

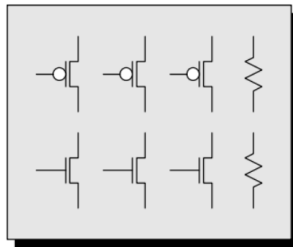


- Logic block, SRAM block, DSP block, PLL, Clock manager, I/O bank, SERDES block.

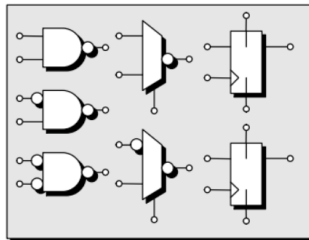
Flujo de diseño típico en FPGA



- ▶ Más de 100 millones de compuertas lógicas
- ▶ Diseño completamente personalizado (Full-custom)
 - Máximo rendimiento
 - Muy costoso
- ▶ Matrices de compuertas (Gate Arrays)
 - Conjunto de celdas estándar no conectadas (transistores, resistencias, MUXs, etc.)
 - Las conexiones se definen al final del proceso de fabricación y son fijas
- ▶ ASICs estructurados
 - Similares a las Gate Arrays, pero con celdas de nivel más alto



Gate Array

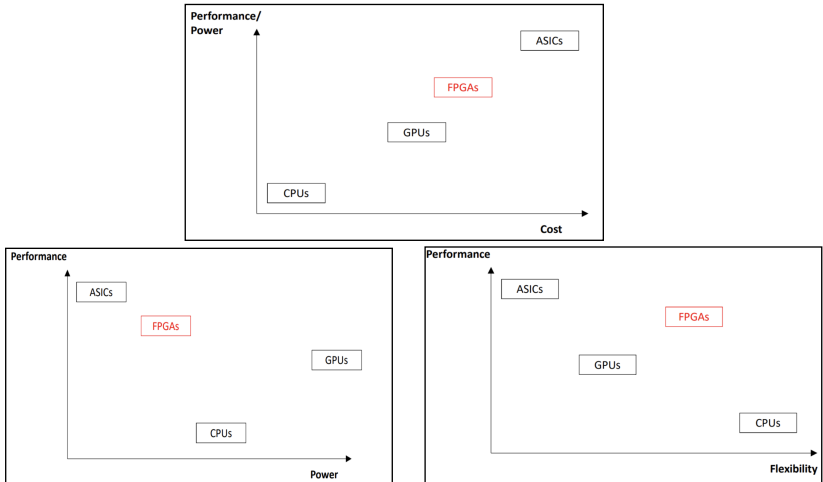


Structured ASICs

Application-Specific Integrated Circuit (ASIC)

- ▶ **ASIC:** Circuito integrado diseñado para una **función específica**.
- ▶ **Características:**
 - Altamente eficiente en **rendimiento y consumo energético**.
 - **Menor flexibilidad:** no se puede reprogramar.
 - Elevado **costo de diseño y fabricación**, pero bajo costo por unidad en producción masiva.
- ▶ **Ejemplos de aplicación:**
 - Minería de criptomonedas.
 - Telecomunicaciones (4G/5G).
 - Chips especializados en smartphones.
 - Sistemas automotrices (airbags).
- ▶ **Comparación con FPGA:**
 - **FPGA:** flexible, configurable, ideal para prototipos y bajas producciones.
 - **ASIC:** óptimo para producción masiva y alto rendimiento.

FPGA vs. Rest of the World



► **Clave para nuevos paradigmas de computación emergentes:**

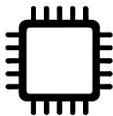
- **Cloud Computing** - Eficiencia energética.
- **Machine Learning** - Nuevas funciones.
- **Computación reconfigurable** - Requisitos y estándares en evolución.

TPU (Tensor Processing Unit)

- ▶ **Unidad de Procesamiento de Tensores** diseñada por Google específicamente para acelerar cargas de trabajo de IA.
- ▶ Optimizada para operaciones de DL, especialmente para *TensorFlow*.
- ▶ **Características**
 - Arquitectura basada en **arrays de multiplicadores-acumuladores** (MACs).
 - Alta paralelización y ancho de banda de memoria optimizado para operaciones matriciales.
 - Bajo consumo energético comparado con GPUs en tareas específicas.

Aplicaciones

- ▶ Inferencia y entrenamiento de modelos de aprendizaje
- ▶ Integración en Google Cloud Platform como servicio (*TPU as a Service*).



CPU

General-purpose & classic ML



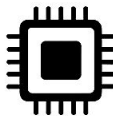
GPU

Parallel deep learning



NPU

On-device edge AI



TPU

Cloud-scale TensorFlow



DPU

AI infrastructure acceleration

► NPU (Neural Processing Unit)

- Chips especializados en operaciones de redes neuronales (ASICs especializados, SoC).
- Usados en smartphones e IoT para IA en tiempo real.
- Optimizados para bajo consumo energético (ej. reconocimiento facial, traducción en el dispositivo).
- Ejecutan modelos ligeros y eficientes (MobileNet, TinyBERT).

► TPU (Tensor Processing Unit)

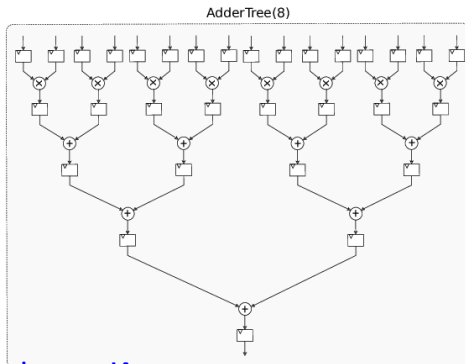
- Diseñados para entrenar y desplegar redes neuronales masivas (Clústeres).
- Alta velocidad en cálculos tensoriales.
- Ideales para modelos grandes en la nube (BERT, GPT-2, EfficientNet).

► DPU (Data Processing Unit)

- No ejecutan modelos de IA directamente.
- Aceleran el movimiento de datos, redes y almacenamiento.
- Liberan CPU y GPU para cómputo intensivo.
- Clave en infraestructura moderna de IA a gran escala.

Métricas de rendimiento y eficiencia en HPC para IA

Métricas de rendimiento y eficiencia en HPC para IA



Latencia y Throughput en IA

- **Latencia:** tiempo que tarda en obtenerse un resultado desde la entrada → *crítico en sistemas de tiempo real*
- **Throughput:** número de inferencias procesadas por segundo → clave en *despliegues masivos* (ej. servidores de IA).

FLOPS y eficiencia energética

- FLOPS: número de operaciones de punto flotante por segundo.
- Métrica tradicional de HPC, pero **no suficiente** para IA moderna.

¿Qué es y cómo se mide FLOPS?

- **FLOP** (*Floating Point Operation*) = una operación aritmética con números en punto flotante.

Ejemplo: $c = a \times b + d$ cuenta como 2 FLOPs.

- **FLOPS** (*Floating Point Operations per Second*) = tasa de operaciones de punto flotante por segundo.

Cálculo teórico

$$\text{FLOPS} = (\text{CUDA cores (or SMs)}) \times (\text{operaciones por ciclo}) \times (\text{frec or clock speed})$$

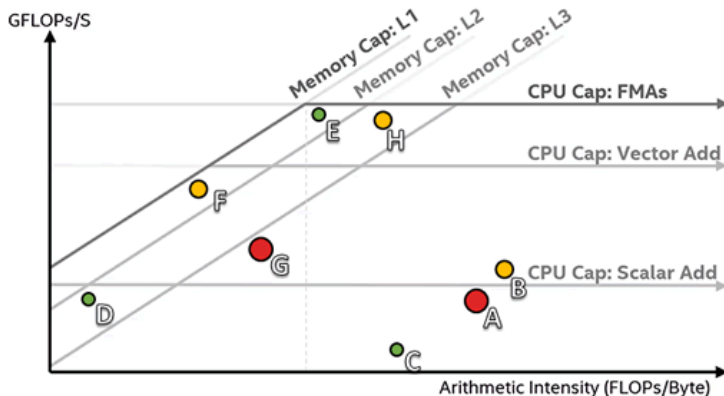
Ejemplo:

- GPU con 5120 CUDA cores, 2 operaciones/ciclo, 1.5 GHz:

$$\text{FLOPS} = 5120 \times 2 \times 1,5 \times 10^9 \approx 15,4 \text{ TFLOPS}$$

- **FLOPS peak:** máximo teórico bajo condiciones ideales.
- **FLOPS** medidos en aplicaciones reales están limitados por memoria y latencia.

Modelo Roofline



- ▶ Herramienta gráfica que relaciona el **rendimiento computacional (FLOPS)** con el **ancho de banda de memoria**.
- ▶ Permite identificar si una aplicación está limitada por **cómputo (compute-bound)** o por **memoria (memory-bound)**.
- ▶ Útil para detectar cuellos de botella y guiar optimizaciones¹⁶.

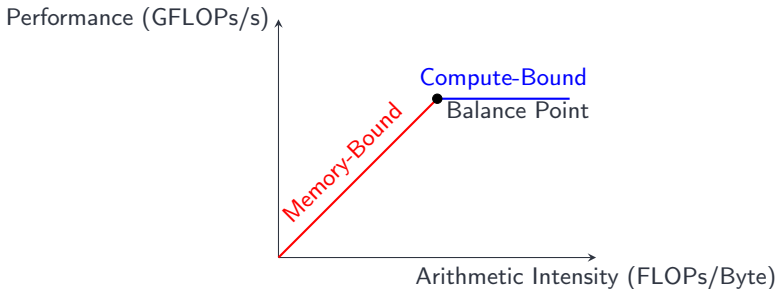
¹⁶(vectorización (aplicar la misma operación a varios datos al mismo tiempo, SIMD), tiling (reutilizar datos ya cargados en caché, evitando accesos costosos a memoria principal), uso de memoria compartida)

Modelo Roofline

Arithmetic Intensity (AI): $AI = \frac{\text{FLOPs}}{\text{Bytes Movidos}}^{17}$

Límite de Rendimiento: $P = \min(P_{\text{peak}}, AI \times BW_{\text{mem}})$

- ▶ Valores bajos de AI: **Memory-Bound** → limitado por el ancho de banda¹⁸.
- ▶ Valores altos de AI: **Compute-Bound** → limitado por la capacidad de cómputo¹⁹.
- ▶ El cruce define el **Balance Point**.



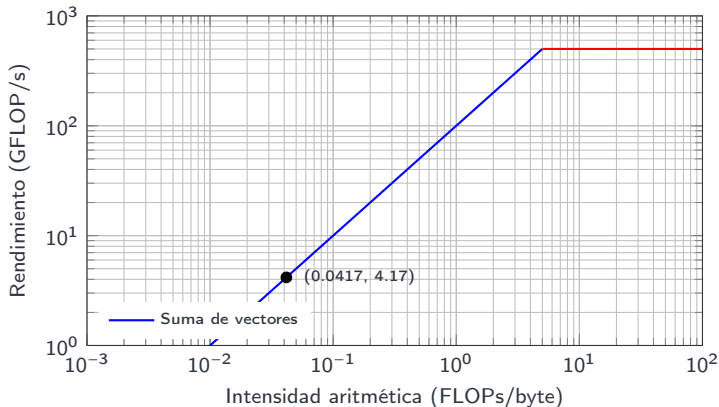
¹⁷cuántos datos deben moverse desde la memoria (RAM o caché)

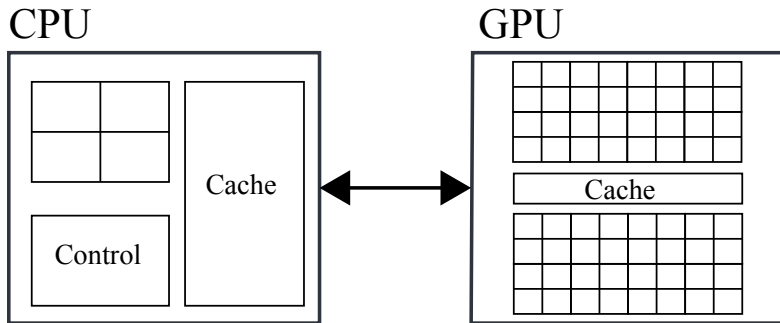
¹⁸Pocos cálculos por cada byte movido. Recorrer vector sumando

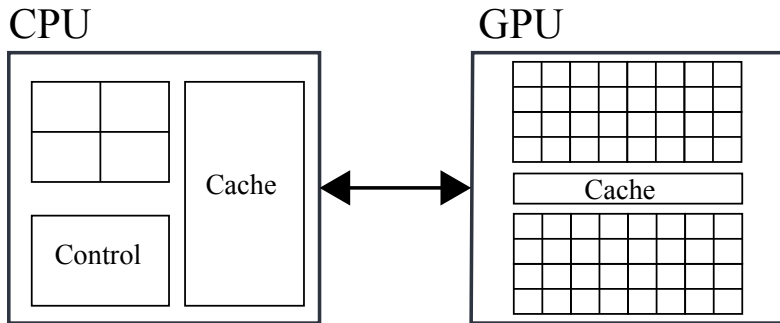
¹⁹multiplicaciones de matrices grandes → se reutilizan datos cargados en memoria varias veces

Roofline - Suma de dos vectores

- ▶ Operación: $C[i] = A[i] + B[i]$ (1 FLOP por elemento).
- ▶ Tráfico de memoria (en double): leer A (8 B) + B (8 B) y escribir C (8 B) $\Rightarrow$ 24 B/elem.
- ▶ Intensidad aritmética: $IA = \frac{\text{FLOPs}}{\text{Bytes}} = \frac{1}{24} \approx 0,0417$ FLOPs/byte.
- ▶ Ejemplo: pico de cómputo = 500 GFLOP/s y ancho de banda = 100 GB/s.
- ▶ Techo por memoria: $100 \text{ GB/s} \times 0,0417 \approx 4,17 \text{ GFLOP/s} \Rightarrow$ **limitado por memoria**.





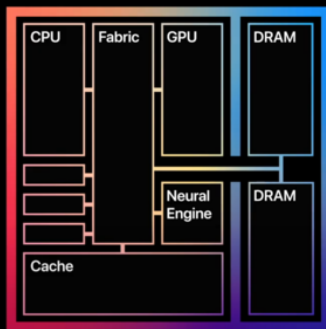


AMD
XILINX

System-on-chip (SoC)

System-on-chip (SoC)

Apple M1



Unified memory architecture

High bandwidth, low latency
Apple-designed package
Accessible to entire SoC

SoC típico que integra CPU, GPU, memoria y controladores de E/S **en un solo chip**. La CPU gestiona las tareas generales, mientras que la GPU se encarga del procesamiento paralelo. Ambos comparten acceso a la memoria a través del controlador DRAM.

Multiprocessor System-on-Chip (MPSoC)

Un **Multiprocessor System-on-Chip (MPSoC)** integra múltiples núcleos de procesamiento heterogéneos (CPU, GPU, DSP, NPU, etc.) en un único chip. Gracias a su arquitectura paralela y al uso de interconexiones internas, permite ejecutar tareas de manera eficiente y simultánea.

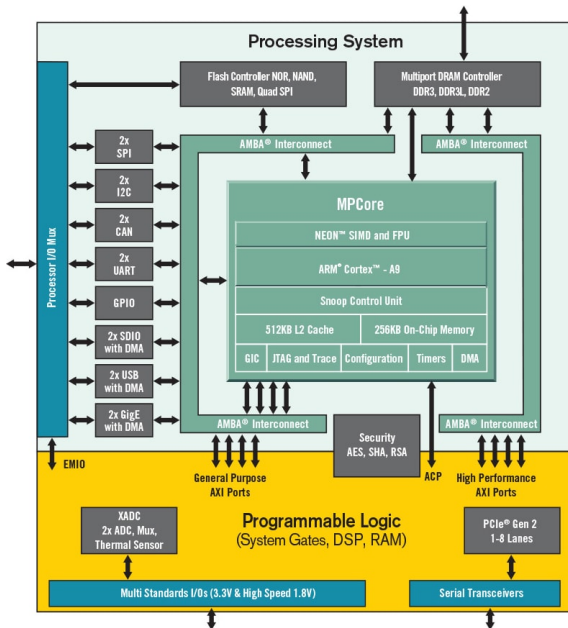
Características:

- ▶ Integración de **CPU** para control general y lógica secuencial.
- ▶ Inclusión de **GPU / NPU** para procesamiento paralelo y redes neuronales.
- ▶ Soporte de **DSP** para tareas de señal, audio y video.
- ▶ Alta eficiencia energética y **reducción de latencia por comunicación interna**.

Aplicaciones:

- ▶ IA y ML.
- ▶ Sistemas embebidos en IoT y dispositivos móviles.
- ▶ Automoción, visión computacional y telecomunicaciones.

AMD Xilinx Zynq System-on-chip



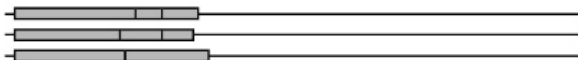
Modelos de programación paralela

¿Qué significa “programar para el paralelismo”?

Concepts in Concurrency



Concurrent, non-parallel execution



Concurrent, parallel execution

El paralelismo consiste en dividir tareas grandes en subtarear pequeñas que pueden ejecutarse simultáneamente en múltiples núcleos o hilos.

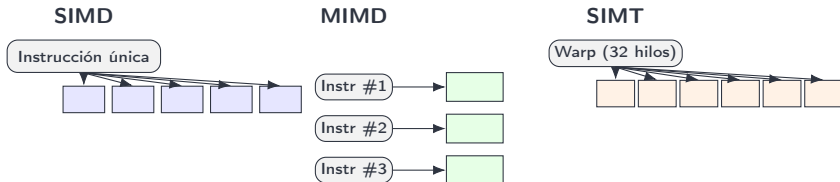
- ▶ Mejora del rendimiento mediante ejecución concurrente.
- ▶ Pensamiento no secuencial: múltiples hilos activos al mismo tiempo.
- ▶ Clave en arquitecturas modernas como GPU.

Paralelismo de datos y tareas

- ▶ Misma operación, diferentes datos (eg., suma elemento a elemento).
- ▶ Operaciones distintas en paralelo (eg., preprocesar mientras se entrena).

Modelos de ejecución

- ▶ **SIMD**: Single Instruction, Multiple Data (vectorización).
- ▶ **MIMD**: Multiple Instruction, Multiple Data (CPU multinúcleo).
- ▶ **SIMT**: Single Instruction, Multiple Threads (CUDA).



Límites teóricos del paralelismo:

Speedup

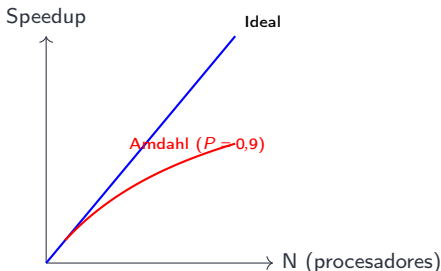
$$S_p = \frac{T_1}{T_p}$$

- ▶ T_1 : tiempo en un procesador; T_p : tiempo en p procesadores.
- ▶ Speedup ideal: $S_p = N$ (lineal).

Ley de Amdahl

$$S_p = \frac{1}{(1 - P) + \frac{P}{N}}$$

- ▶ P : fracción paralelizable del programa; N : # de procesadores.
- ▶ Ejemplo: con $P = 0,9$ y $N = 8$, $S_p \approx 5,26$ (no llega a 8 por la parte secuencial).



Memoria compartida (multihilo)

- ▶ `pthread`s, `std::thread` (C++), **OpenMP**.
- ▶ Comunicación mediante variables compartidas y sincronización.

Paralelismo masivo

- ▶ **CUDA**, OpenCL: miles de hilos simultáneos (SIMT).

	CPU multi-core	GPU CUDA
Hilos	decenas	miles a millones
Memoria	cachés profundas	shared/global/registros
Modelo	MIMD + SIMD	SIMT (warps)
Facilidad	alta (OpenMP)	requiere mapeo explícito

Suma de vectores en paralelo con OpenMP (C++)

```
1 // Compilar: g++ -O3 -fopenmp sum_vec.cpp -o sum_vec
2 #include <omp.h>
3 #include <iostream>
4 #include <vector>
5
6 int main() {
7     const int N = 10'000'000;
8     std::vector<float> A(N, 1.0f), B(N, 2.0f), C(N);
9
10    #pragma omp parallel for
11    for (int i = 0; i < N; ++i) {
12        C[i] = A[i] + B[i];
13    }
14
15    std::cout << "C[0] = " << C[0] << "\n";
16    return 0;
17 }
```

- ▶ `#pragma omp parallel for` divide el bucle entre hilos.
- ▶ Este patrón se migra a CUDA cambiando el mapeo de índices (hilo → elemento).

#pragma es una directiva para el compilador. En OpenMP, se usa para indicar regiones de código que deben ejecutarse en paralelo.

Tipos comunes de #pragma omp

- ▶ `parallel` — Inicia una región paralela.
- ▶ `parallel for` — Paraleliza un bucle `for`.
- ▶ `sections` — Divide el trabajo en secciones independientes.
- ▶ `single` — Solo un hilo ejecuta esa sección.
- ▶ `critical` — Sección protegida contra condiciones de carrera.
- ▶ `atomic` — Operaciones simples sin condiciones de carrera.
- ▶ `barrier` — Sincronización de todos los hilos.

OpenMP Directives: <https://learn.microsoft.com/es-es/cpp/parallel/openmp/reference/openmp-directives?view=msvc-170>

Modelos de programación en CPU (memoria compartida)

- ▶ **OpenMP** (C, C++, Fortran)
 - API de alto nivel para paralelizar bucles con `#pragma omp`.
 - Ideal para tareas científicas o numéricas.
- ▶ **pthread / std::thread** (C/C++)
 - Bajo nivel (pthread), moderno (std::thread).
 - Control total del manejo de hilos y sincronización.
- ▶ **Intel TBB (Threading Building Blocks)**
 - Librería C++ para tareas paralelas, balance de carga, sincronización eficiente.
- ▶ **Python:**
 - `multiprocessing`, `concurrent.futures`, `joblib`
 - Numba con `@jit(parallel=True)` y `prange()`

Consideraciones: uso de memoria, balance de carga, sincronización y portabilidad.

<https://colab.research.google.com/drive/13Fhg43Y5pxXUQntJtH4AC9b1u1xX9cjk#scrollTo=AZTcAjsxVufrJ>

Del paralelismo en CPU al paralelismo en GPU

CPU Multi-core

- Pocos hilos, más trabajo por hilo (OpenMP).

GPU CUDA

- Miles de hilos, menos trabajo por hilo: `threadIdx`, `blockIdx`, `blockDim`.

CPU (OpenMP)

- Bucle for paralelo.
- Sincronización implícita al final del bucle.

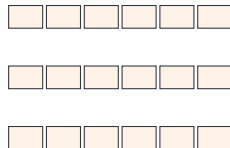
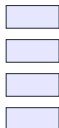
GPU (CUDA)

- Kernel en rejilla de bloques $\times$ hilos.

- Índice global:

$$i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

CPU (4 hilos) $\xrightarrow{\text{mismo algoritmo, distinto mapeo}}$ GPU (bloques $\times$ hilos)

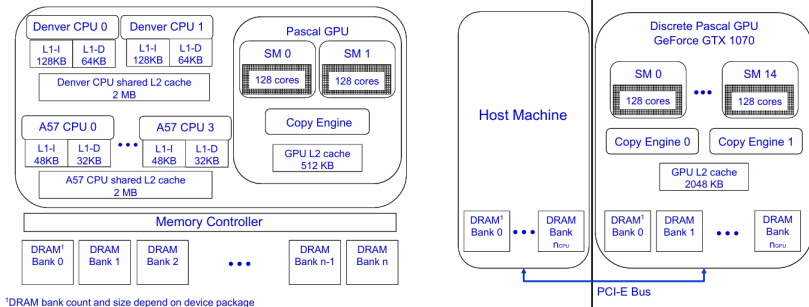


CUDA y OpenCL son frameworks para computación paralela, con distintos niveles de especialización y portabilidad.

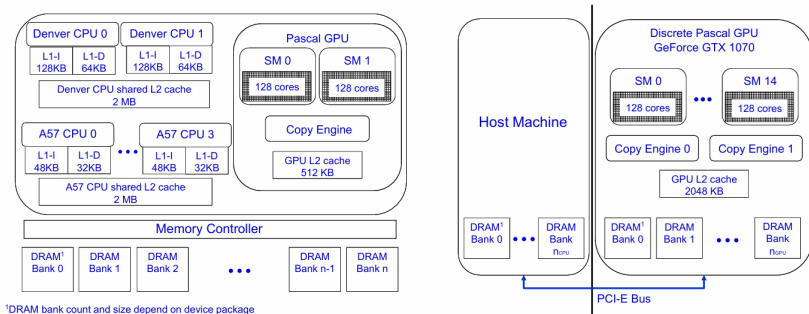
Característica	CUDA	OpenCL
Aplicación	NVIDIA	Multiplataforma
Lenguaje base	Extensión de C/C++	Basado en C99
Portabilidad	Limitada (hardware NVIDIA)	Alta (múltiples vendors)
Acceso al hardware	Directo y optimizado	Generalizado

- ▶ De procesadores gráficos a plataformas de cómputo general
- ▶ CUDA y OpenCL permiten programar la GPU para cómputo paralelo, pero con enfoques distintos
- ▶ Las GPU son importantes para el futuro de la IA y la computación científica.

Diferentes tipos de memoria ofrecen distintos niveles de acceso y velocidad.



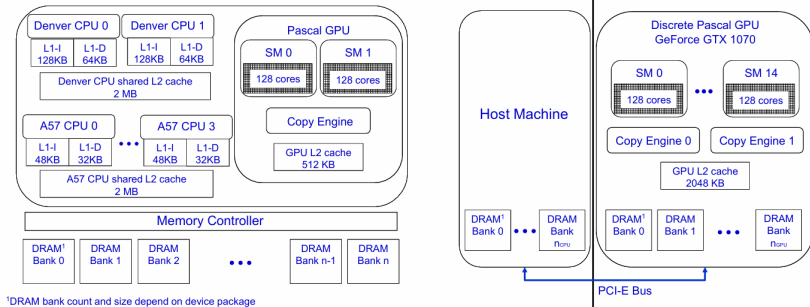
- **Global memory:** Accesible por todos los hilos; lenta.
- **Shared memory:** Compartida entre hilos de un mismo bloque; rápida.
- **Local memory:** Privada por hilo; temporal.



- Diagrama de comunicación entre *host* y *device*: GPU integrada (izquierda) y GPU discreta (derecha)²⁰
- CUDA es un modelo de programación **heterogéneo** que utiliza tanto la CPU como la GPU.

²⁰Ming Yang, et al, "Avoiding Pitfalls when Using NVIDIA GPUs for Real-Time Tasks in Autonomous Systems," *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2021.

Elementos del Procesamiento Paralelo en GPUs



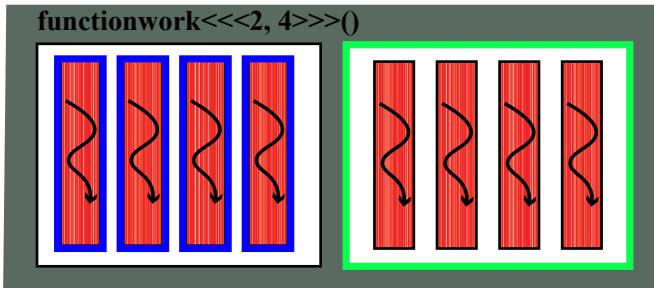
► Implementación *system-on-chip* (SoC).

- Esta integra la GPU, los *CPU Cores*, la memoria (*Unified Memory*) y los conectores externos en una pequeña computadora de una sola placa. La GPU integrada comparte recursos de hardware, como la memoria, con los *CPU Cores*.

► GPU discreta

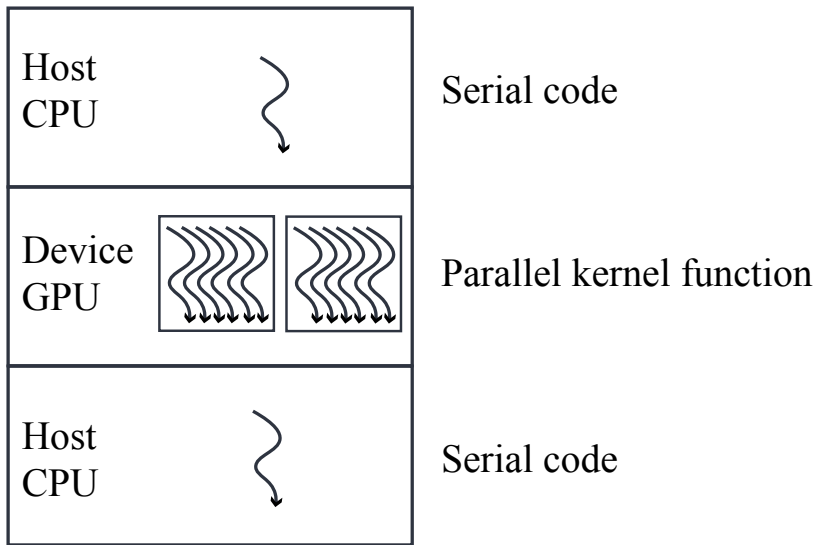
- GPUs discretas, consisten únicamente en los *Streaming Multiprocessors* (SMs) y la memoria local del dispositivo (*device memory*).
- Empaquetados en una tarjeta para su instalación en una ranura de expansión PCIe de la placa base de una computadora.

GPU



- ▶ Un *kernel* es una función que corre en la GPU.
- ▶ Se lanza sobre una **rejilla de bloques**, cada uno con múltiples hilos.
- ▶ Cada hilo tiene identificadores: `threadIdx`, `blockIdx`, etc.

Ejemplo: Rejilla 2x2 bloques, cada bloque con 4x4 hilos $\Rightarrow$ 64 hilos totales.



```
1  __global__ void sumaVectorial(float *A, float *B, float *C, int N) {  
2      int i = threadIdx.x + blockDim.x * blockIdx.x;  
3      if (i < N) C[i] = A[i] + B[i];  
4  }  
5  
6  int main() {  
7      // Asignar memoria, copiar datos, lanzar kernel...  
8      sumaVectorial<<<gridSize, blockSize>>>(A, B, C, N);  
9  }
```

- ▶ `__global__` indica que es un kernel.
- ▶ `<<< ... >>>` define la configuración de ejecución.

Desafíos de HPC en IA



- ▶ **Desbalance entre cómputo y comunicación:** los cuellos de botella no siempre están en la GPU, sino en la red o el almacenamiento compartido.
- ▶ **Escalabilidad limitada en entrenamiento distribuido:** sincronización de gradientes, latencia entre nodos y eficiencia de batch-size afectan el rendimiento final.
- ▶ **Optimización para inferencia:** en producción, la prioridad cambia: latencia, consumo energético y throughput por vatio se vuelven críticos.
- ▶ **TinyML y HPC embebido:** modelos ligeros requieren aceleración en dispositivos edge, lo que impone desafíos de diseño hardware-software.
- ▶ **Herramientas avanzadas:** frameworks como TensorRT, Vitis AI y plataformas como Jetson habilitan inferencia eficiente en entornos de recursos limitados.

Sostenibilidad y huella energética en HPC para IA

- ▶ Entrenar modelos masivos (ej. GPT-3, GPT-4) requiere **miles de GPUs durante semanas**.
- ▶ Consumo energético comparable al de **ciudades pequeñas**.
- ▶ Costos de entrenamiento: **cientos de millones de dólares**.
- ▶ Impacto ambiental: **huella de carbono significativa** por uso intensivo de centros de datos.

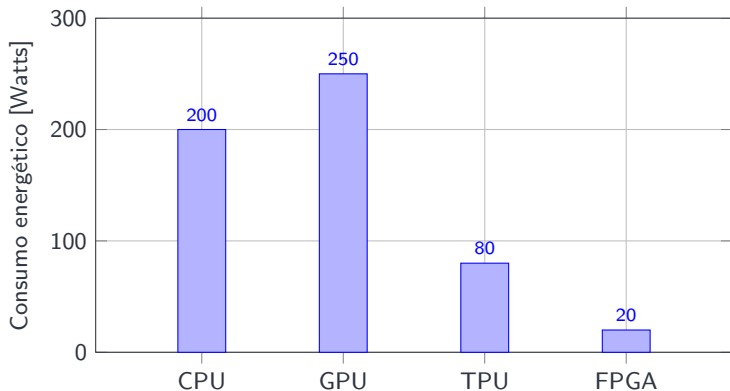
Green AI

- ▶ Enfoque que prioriza **eficiencia energética y de recursos**, además de precisión.
- ▶ Modelos competitivos con **menor costo computacional**.
- ▶ Indicadores:
 - **perf/Watt** (rendimiento por vatio).
 - **CO emissions per training run**.

Técnicas de optimización

- ▶ **Quantization**: FP32 $\rightarrow$ INT8.
- ▶ **Pruning**: eliminar conexiones/pesos redundantes.
- ▶ **Knowledge Distillation**: entrenar modelos pequeños basados en grandes.
- ▶ **Hardware especializado**: TPUs, FPGAs.

Sostenibilidad y Huella Energética en HPC para IA



Observaciones

- ▶ **CPU**: alta versatilidad, pero menor eficiencia energética.
- ▶ **GPU**: alto rendimiento, pero consumo elevado.
- ▶ **TPU**: optimizadas para IA, mejor **perf/Watt**.

Entornos HPC

- ▶ **SLURM**: gestor de colas y planificación en supercomputadoras.
- ▶ **Kubernetes adaptado a IA**: orquestación de contenedores con soporte para cargas de DL.

Contenedores en HPC

- ▶ **Singularity**: diseñado para entornos HPC, sin privilegios de root.
- ▶ **Docker para IA**: integración con frameworks como TensorFlow y PyTorch.

Frameworks distribuidos

- ▶ **Horovod**: entrenamiento distribuido eficiente con TensorFlow y PyTorch.
- ▶ **DeepSpeed**: optimización de modelos grandes (billones de parámetros).
- ▶ **Ray**: plataforma flexible para aprendizaje distribuido y tareas paralelas.

Inference Optimizer

- ▶ Reducir **latencia**, **consumo energético** y mejorar **throughput** en la inferencia de modelos de IA.
- ▶ Adaptar modelos entrenados a hardware específico para ejecución en producción.

Herramientas

- ▶ **NVIDIA TensorRT**: optimización de redes profundas en GPUs.
- ▶ **Xilinx Vitis AI**: inferencia en FPGAs y SoCs.
- ▶ **Intel OpenVINO**: optimización en CPUs, VPUs y FPGAs.
- ▶ **ARM Compute Library**: optimización en dispositivos móviles y embebidos.

Técnicas

- ▶ **Quantization**: reducción de precisión (FP32 $\rightarrow$ INT8).
- ▶ **Pruning**: eliminación de pesos redundantes.
- ▶ **Fusión de operadores**: combinar capas para reducir accesos a memoria.
- ▶ **Batching**: procesamiento de múltiples entradas en paralelo.

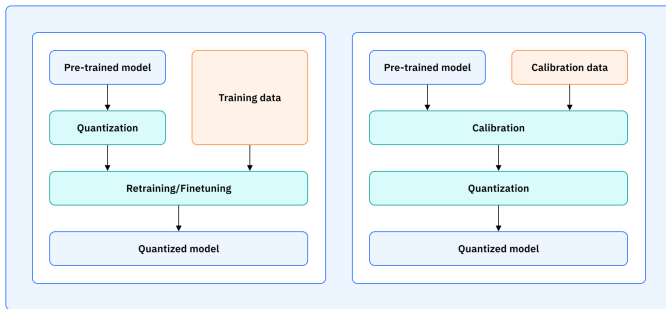
La cuantización es una técnica clave en HPC para IA, que reduce la precisión de los parámetros y activaciones de un modelo (ej. de FP32 a INT8), con el objetivo de mejorar el rendimiento y disminuir el consumo energético.

- ▶ Reducción del tamaño del modelo.
- ▶ Menor uso de memoria y ancho de banda.
- ▶ **Inferencia más rápida**, especialmente en hardware especializado (GPU, TPU, NPU).
- ▶ Menor consumo energético en dispositivos embebidos y móviles.

Métodos

- ▶ **Post-Training Quantization (PTQ)**: Se aplica después del entrenamiento. Rápida, pero puede perder algo de precisión.
- ▶ **Quantization-Aware Training (QAT)**:²¹ La cuantización se realiza durante el entrenamiento. Mayor precisión, ideal para modelos grandes.

²¹<https://pytorch.org/blog/quantization-aware-training/>



Quantization-Aware Training (QAT)

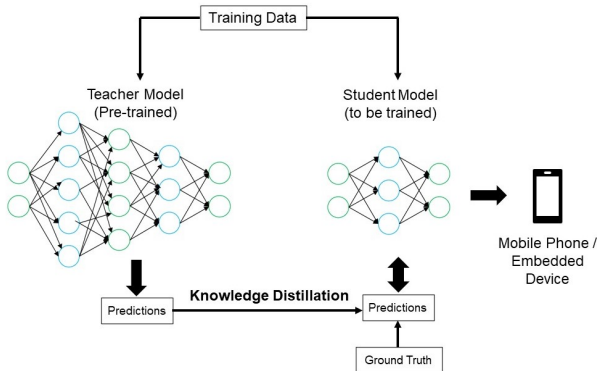
- ▶ Integra la cuantización durante el entrenamiento.
- ▶ Mantiene mayor precisión.
- ▶ **Requiere dataset representativo.**
- ▶ Alto costo computacional.

Post-Training Quantization (PTQ)

- ▶ Se aplica después del entrenamiento.
- ▶ **No requiere dataset original.**
- ▶ Fácil y rápida de implementar.
- ▶ Puede perder algo de precisión.

Knowledge distillation

KD is a machine learning technique that aims to transfer the learnings of a large pre-trained model, the “teacher model,” to a smaller “student model” (*light-weight neural network*). It’s used in deep learning as a form of model compression and knowledge transfer, particularly for massive deep neural networks.



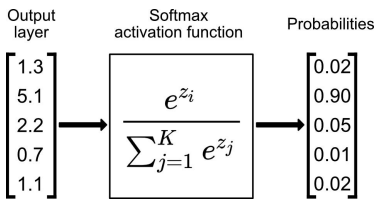
Training and inference have difference requirements

Knowledge Distillation y Softmax con Temperatura

- ▶ **Teacher Model:** Produce una distribución suave.
- ▶ **Student Model:** Se entrena con estas distribuciones suaves (soft targets) en lugar de etiquetas duras.
- ▶ **Ventaja:** Captura patrones más ricos y generalizables sin perder información de clases cercanas.

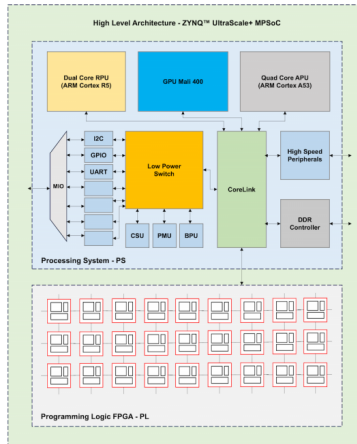
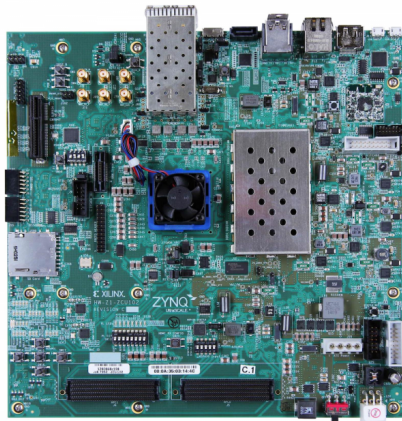
Knowledge Distillation y Softmax con Temperatura

- **Teacher Model:** Produce una distribución suave.
- **Student Model:** Se entrena con estas distribuciones suaves (soft targets) en lugar de etiquetas duras.
- **Ventaja:** Captura patrones más ricos y generalizables sin perder información de clases cercanas.

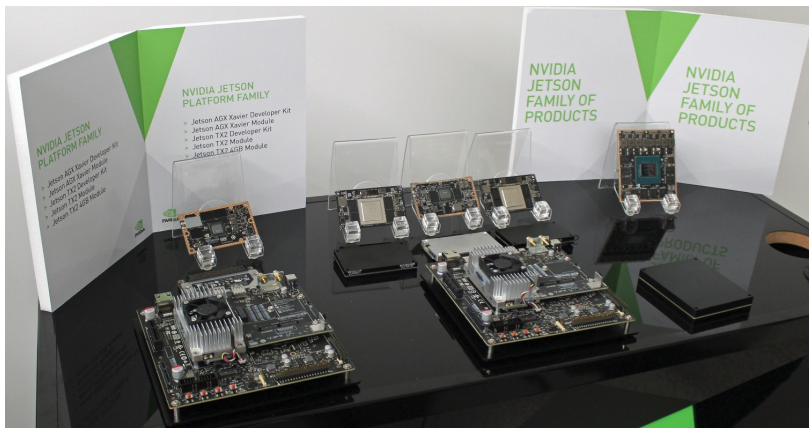


- La temperatura elevada permite que el *teacher model* produzca *soft targets* más suaves.
- El *student model* se ajusta a esos *soft targets*, aprendiendo sobre relaciones entre clases.

Zynq UltraScale+ MPSoC ZCU104 Evaluation Kit



Jetson - Características de los dispositivos



Plataforma	Núcleos GPU	Memoria (GB)	CPU
Nvidia GeForce RTX 3060	—	12 (GDDR6) + 36 RAM	Intel i3-12100F
Nvidia GeForce RTX 2060	—	6 (GDDR6) + 12 RAM	Intel i7-10750H
Jetson Xavier NX	384	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.9 GHz
Jetson Xavier AGX	512	16 RAM (compartida)	ARMv8.2, 8 núcleos, 2.3 GHz
Jetson Orin Nano	1024	8 RAM (compartida)	ARMv8.2, 6 núcleos, 1.7 GHz
Jetson Orin AGX	2048	64 RAM (compartida)	ARMv8.2, 12 núcleos, 2.2 GHz

Niveles populares de software y hardware.

Capa	Ejemplos
Aplicación	Visión por computadora Control Automático Reconocimiento de voz Conducción autónoma
Modelo	ResNet - MobileNet - MobileViT YOLO - U-Net
Framework	PyTorch - TensorFlow - Keras
Graph Format	ONNX NNEF ²⁴
Inference Optimizer	TensorRT ARM Compute Library Intel OpenDNN AMD ML Open Xilinx Vitis AI OpenVINO Toolkit
Hardware	GPU CPU FPGA Aceleradores especializados (TPU, NPU, etc.)

²⁴Sintaxis similar a JSON o Python, lo que facilita su comprensión y edición manual

Material de estudio:

- ▶ CUDA Programming Guide (NVIDIA)
- ▶ NVIDIA TensorRT
- ▶ TensorRT Documentation
- ▶ Tesla V100 Performance Guide
- ▶ Arquitectura Hopper
- ▶ NVIDIA Multi-Instance GPU

Revisión de arquitecturas: GPU vs CPU vs FPGA.

- ▶ P. Dhilleswararao, S. Boppu, M. S. Manikandan and L. R. Cenkeramaddi, 'Efficient Hardware Architectures for Accelerating Deep Neural Networks: Survey' in IEEE Access, vol. 10, pp. 131788-131828, 2022, doi: 10.1109/ACCESS.2022.3229767.



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Introducción a HPC para IA

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile